Tuple

In [1]:
```
# Empty Tuple
t1 = ()
t2 = tuple()
```

In [3]:
```
print(type(t1))
print(type(t2))
```

```
<class 'tuple'>
<class 'tuple'>
```

In [4]:
```
print(t1, t2)
```

```
() ()
```

In [7]:
```
# List having one element
li = [5]
print(li)
print(type(li))
```

```
[5]
<class 'list'>
```

In [11]:
```
# Tuple having one element
t = (5)
print(t)
print(type(t))
```

```
5
<class 'int'>
```

In [13]:
```
t = (5,)
print(t)
print(type(t))
```

```
(5,)
<class 'tuple'>
```

In [15]:
```
t3 = (10, 8.9, True, 6+7j, "Hi")
```

In [17]:
```
print(t3)
```

```
(10, 8.9, True, (6+7j), 'Hi')
```

In [19]:
```
t3[0]
```

Out[19]:
```
10
```

In [21]:
```
t3[0] = 100 #as it is immutable
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 t3[0] = 100

TypeError: 'tuple' object does not support item assignment
```

In [23]:
```
t4 = tuple(range(1,6))
t5 = tuple("python")
t6 = tuple([11,22,33])
```

In [25]:
```
print(t4,t5,t6, sep = "\n")
```

```
(1, 2, 3, 4, 5)
('p', 'y', 't', 'h', 'o', 'n')
(11, 22, 33)
```

Tuple Packing

In [28]:
```
t1 = 1,2,3,4,5 #assignment of multiple values to one object
```

In [30]:
```
print(t1)
print(type(t1))
```

```
(1, 2, 3, 4, 5)
<class 'tuple'>
```

Tuple Unpacking

In [33]:
```
a,b,c,d,e = t1
```

In [35]:
```
print(a,b,c,d,e)
```

```
1 2 3 4 5
```

In [37]:
```
a,b,c = t1
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[37], line 1
----> 1 a,b,c = t1

ValueError: too many values to unpack (expected 3)
```

In [39]:
```
a,b,*c = t1
print(a,b,c)
```

```
1 2 [3, 4, 5]
```

In [41]:
```
a,*b,c = t1
print(a,b,c)
```

```
1 [2, 3, 4] 5
```

In [43]:
```
*a,b,c = t1
print(a,b,c)
```

```
[1, 2, 3] 4 5
```

In [45]:
```
a,*b,*c = t1
print(a,b,c)
```

```
  Cell In[45], line 1
    a,*b,*c = t1
    ^
SyntaxError: multiple starred expressions in assignment
```

In [47]:
```
l1 = [11,22,33,44,55] #packing happens for tuple only, but any iterable object c
```

In [49]:
```
a,b,*c = l1
```

In [51]:
```
print(a,b,c)
```

```
11 22 [33, 44, 55]
```

In [53]:
```
s = "Python"
a,b,*c = s
print(a,b,c)
```

```
P y ['t', 'h', 'o', 'n']
```

In [55]:
```
t3
```

Out[55]:
```
(10, 8.9, True, (6+7j), 'Hi')
```

Tuple Methods : count(element), index(element, start, stop)

In [58]:
```
t = (10,20,30,20,40,50,40)
```

In [60]:
```
t.count(40)
```

Out[60]:
```
2
```

In [62]:
```
t.index(20)
```

Out[62]:
```
1
```

In [64]:
```
t.index(20,2)
```

Out[64]:
```
3
```

Tuple Comprehension

In [67]:
```
[ i for i in range(1,6)]
```

Out[67]:
```
[1, 2, 3, 4, 5]
```

In [69]:
```
(i for i in range(1,6))
```

Out[69]:
```
<generator object <genexpr> at 0x00000229FF26BC40>
```

In [71]:
```
tuple(i for i in range(1,6)) #method-1
```

Out[71]:
```
(1, 2, 3, 4, 5)
```

In [73]:
```
*(i for i in range(1,6)), #method-2
```

Out[73]:
```
(1, 2, 3, 4, 5)
```

Set

In [76]:
```
#This will create empty dictionary
s1 = {}
print(s1)
print(type(s1))
```

```
{}
<class 'dict'>
```

In [78]:
```
#only way to create an empty set
s1 = set()
print(s1)
print(type(s1))
```

```
set()
<class 'set'>
```

In [80]:
```
s2 = {10, 7.6, False, 9+8j, "Hi"}
```

In [82]:
```
print(s2)
```

```
{False, (9+8j), 'Hi', 7.6, 10}
```

In [84]:
```
#unordered collection, so indexing is not allowed
s2[0]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[84], line 2
      1 #unordered collection, so indexing is not allowed
----> 2 s2[0]

TypeError: 'set' object is not subscriptable
```

In [86]:
```
#can be iterated using for each loop
for x in s2:
    print(x)
```

```
False
(9+8j)
Hi
7.6
10
```

In [88]:
```
#index based looping is not possible
for i in range(len(s2)):
    print(s2[i])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[88], line 3
      1 #index based looping is not possible
      2 for i in range(len(s2)):
----> 3     print(s2[i])

TypeError: 'set' object is not subscriptable
```

In [90]:
```
print(s2)
```

```
{False, (9+8j), 'Hi', 7.6, 10}
```

In [92]:
```
#mutable, but can't change the value using index as it is unordered as well
s2[4] = 100
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[92], line 2
      1 #mutable, but can't change the value using index as it is unordered as we
ll
----> 2 s2[4] = 100

TypeError: 'set' object does not support item assignment
```

In [94]:
```
# To change value of any element
s2.add(100)
```

In [96]:
```
s2
```

Out[96]:
```
{(9+8j), 10, 100, 7.6, False, 'Hi'}
```

In [98]:
```
s2.remove(10)
```

In [100…
```
s2
```

Out[100…
```
{(9+8j), 100, 7.6, False, 'Hi'}
```

In [110…
```
# mutable objects can't be set element
s2.add([10,20])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[110], line 2
      1 # mutable objects can't be set element
----> 2 s2.add([10,20])

TypeError: unhashable type: 'list'
```

In [112…
```
s2.add((10,20))
```

In [114…
```
s2
```

Out[114…
```
{(10, 20), (9+8j), 100, 7.6, False, 'Hi'}
```

In [116…
```
s2.add({10,20})
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[116], line 1
----> 1 s2.add({10,20})

TypeError: unhashable type: 'set'
```

Set Methods : add(element), update(iterable), copy()

In [119…
```
s2
```

Out[119…
```
{(10, 20), (9+8j), 100, 7.6, False, 'Hi'}
```

In [121…
```
s2.add(40)
```

In [123…
```
s2
```

Out[123…
```
{(10, 20), (9+8j), 100, 40, 7.6, False, 'Hi'}
```

In [125…
```
s2.add("python")
```

In [127…
```
s2
```

Out[127…
```
{(10, 20), (9+8j), 100, 40, 7.6, False, 'Hi', 'python'}
```

In [129…
```
s2.update("python")
```

In [131…
```
print(s2)
```

```
{False, 't', 7.6, (10, 20), 'Hi', 'h', 'n', (9+8j), 100, 40, 'y', 'o', 'python',
'p'}
```

In [133…
```
s2.update(78)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[133], line 1
----> 1 s2.update(78)

TypeError: 'int' object is not iterable
```

In [135…
```
s3 = s2.copy()
```

In [137…
```
print(s3)
```

```
{False, (9+8j), 't', 100, 7.6, 40, (10, 20), 'Hi', 'h', 'y', 'o', 'python', 'p',
'n'}
```

Set Methods : pop(), remove(), discard(), clear()

In [140…
```
print(s3)
```

```
{False, (9+8j), 't', 100, 7.6, 40, (10, 20), 'Hi', 'h', 'y', 'o', 'python', 'p',
'n'}
```

In [142…
```
s3.pop()
```

Out[142…
```
False
```

In [144…
```
s3.pop()
```

Out[144…
```
(9+8j)
```

In [146…
```
s3.pop()
```

Out[146…
```
't'
```

In [148…
```
s3
```

Out[148…
```
{(10, 20), 100, 40, 7.6, 'Hi', 'h', 'n', 'o', 'p', 'python', 'y'}
```

In [150…
```
s3.remove('Hi')
print(s3)
```

```
{100, 7.6, 40, (10, 20), 'h', 'y', 'o', 'python', 'p', 'n'}
```

In [152…
```
s3.remove('Hi') #generates error if element is not avaialble in the set
print(s3)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[152], line 1
----> 1 s3.remove('Hi') #generates error if element is not avaialble in the set
      2 print(s3)

KeyError: 'Hi'
```

In [154…
```
s3.discard('o')
print(s3)
```

```
{100, 7.6, 40, (10, 20), 'h', 'y', 'python', 'p', 'n'}
```

In [156…
```
s3.discard('Hi') #does nothing if the element is not available in the set
print(s3)
```

```
{100, 7.6, 40, (10, 20), 'h', 'y', 'python', 'p', 'n'}
```

In [158…
```
s3.clear()
```

In [160…
```
print(s3)
```

```
set()
```

Set Methods:

- union(set)
- intersection(set)
- difference(set)
- symmetric_difference(set)
- intersection_update(set)
- difference_update(set)
- symmetric_difference_update(set)

In [163…
```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8,9}
```

In [165…
```
s1.union(s2)
```

Out[165…
```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

In [167…
```
s1 | s2
```

Out[167…
```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

In [169…
```
s1.intersection(s2)
```

Out[169… {4, 5}

In [171…
```
s1 & s2
```

Out[171… {4, 5}

In [173…
```
s1.difference(s2)
```

Out[173… {1, 2, 3}

In [175…
```
s1 - s2
```

Out[175… {1, 2, 3}

In [177…
```
s2.difference(s1)
```

Out[177… {6, 7, 8, 9}

In [179…
```
s2 - s1
```

Out[179… {6, 7, 8, 9}

In [181…
```
s1.symmetric_difference(s2)
```

Out[181… {1, 2, 3, 6, 7, 8, 9}

In [183…
```
s2.symmetric_difference(s1)
```

Out[183… {1, 2, 3, 6, 7, 8, 9}

In [189…
```
s1.intersection(s2) #doesn't change s1
```

Out[189… {4, 5}

In [191…
```
s1
```

Out[191… {1, 2, 3, 4, 5}

In [193…
```
#updates s1 by the intesection set
s1.intersection_update(s2) # or s1 = s1 & s2 or s1 &= s2
```

In [195…
```
s1
```

Out[195… {4, 5}

In [197…
```
s1.difference_update(s2) # or s1 -= s2
```

In [199…
```
s2.difference_update(s1) # or s2 -= s1
```

In [201…
```
s1.symmetric_difference_update(s2) # or s1 ^= s2
```

set comprehension

In [204…
```
li = [-4,-3,-2,-1,0,1,2,3,4]
```

In [206…

```
s3 = {i**2 for i in li}
```

In [208…

```
print(s3)
```

```
{0, 1, 4, 9, 16}
```